

Laborator 3

Vom considera cunoscute următoarele definiții:

```
Inductive Var := a | b | c | x | y | z | n | s | i .
```

```
Inductive AExp :=
```

```
| anum : nat -> AExp  
| aplus : AExp -> AExp -> AExp  
| amul : AExp -> AExp -> AExp.
```

```
Coercion anum : nat >-> AExp.
```

```
Notation "A +' B" := (aplus A B) (at level 60, right associativity).
```

```
Notation "A *' B" := (amul A B) (at level 58, left associativity).
```

Deadline pentru exercițiile propuse: **26 octombrie 2020, ora 14h00.**

Exerciții propuse:

1. (1 pct.) Modificați definiția expresiilor aritmetice (**AExp** de mai sus) astfel încât aceasta să permită utilizarea de variabile (**Var**). Testați modificările făcute pe următoarele cazuri:

```
Check 2 +' (avar n).
```

```
Check n +' 1.
```

```
Check s +' i.
```

Hint: utilizați **Coercion** pentru variabile.

2. (2 pct.) Definiți un tip de date pentru expresii booleene care să includă valorile de adevăr, negații, conjuncții și comparații între expresii aritmetice. Testați modificările făcute pe următoarele cazuri:

```
Check btrue.
```

```
Check bfalse.
```

```
Check ! (x <' 10).
```

```
Check btrue and' (n >' 0).
```

Hint: atenție la notațiile utilizate!

3. (3 pct.) Definiți un tip de date pentru instrucțiunile unui limbaj de programare simplu. Instrucțiunile sunt: atribuirii, bucle (de tip `while`) și secvențe de instrucțiuni. Pentru teste, utilizați cazurile de mai jos. Atenție la notații: acestea vă pot ajuta să determinați forma instrucțiunilor utilizate.

```
Check n ::= 10.
Check s ::= 0.
Check n ::= 10 ;; s ::= 0 ;; i ::= 0.
Check n ::= 10 ;;
      s ::= 0 ;;
      i ::= 0 ;;
      while (i < ' n +' 1) (
        s ::= s +' i ;;
        i ::= i +' 1
      ).
```

Coq Reference Manual: <https://coq.inria.fr/distrib/current/refman/>